Implementation Companion

Python Examples for Note 2 PINN/PIDL Cyber Control

From loss terms to reproducible experiments

Companion to the PINN/PIDL cyber-control guide

Contents

1. Purpose

This companion maps the Note 2 equations to the Python files in this repository. The aim is transparency: each example should make clear which unknown function is represented by a neural network, which residual is minimized, what data are needed, and what outputs should be inspected.

2. File Map

Python file	What it demonstrates
`inverse_pinn_sir_malware.py`	Inverse PINN: learn hidden S/I/R trajectories and unknown SIR parameters from sparse infected-state observations.
`pidl_unknown_mechanism.py`	PIDL correction: keep the known SIR mechanism explicit and learn a missing nonlinear transfer term.
`control_pinn_malware.py`	Direct neural-control PINN: train state and control networks using objective, residual, and initial-condition losses.
`pmp_informed_pinn_malware.py`	PMP-informed PINN: train state, costate, and control networks with state, costate, stationarity, and boundary residuals.
`run_training_iterations.py`	Rebuilds CSV histories, baseline metrics, `OUTPUT_PREVIEW.md`, `training_summary.md`, and the diagnostic figures.

3. Common Implementation Pattern

All four examples follow the same practical pattern:

1. define the cyber propagation model and parameter ranges;
2. choose the neural objects, such as state, correction, control, or costate networks;
3. sample observed data and collocation points;
4. build data, residual, boundary, objective, or PMP losses;
5. train with Adam and log each important loss component separately;
6. validate by inspecting trajectories, residuals, parameter estimates, and baseline comparisons.

4. Inverse PINN

The inverse PINN observes sparse samples of the infected component $I(t)$. The state network $N_\theta(t)$ outputs $\hat{x}(t) = [\hat{S}, \hat{I}, \hat{R}]$, and trainable positive parameters represent $\hat{\beta}$ and $\hat{\gamma}$.

Loss structure.

$$\mathcal{L} = \mathcal{L}_{\text{data}} + w_{\text{ic}}\mathcal{L}_{\text{ic}} + w_{\text{ode}}\mathcal{L}_{\text{ode}}.$$

The data term matches sparse $I(t)$ observations, the initial-condition term anchors $x(0)$, and the ODE residual enforces SIR dynamics at collocation points.

Listing 1: Core inverse-PINN residual idea.

```
x_f = model(t_f)
dxdt = time_derivative(x_f, t_f)
rhs = sir_rhs(x_f, beta, gamma)
loss_ode = torch.mean((dxdt - rhs) ** 2)
loss = loss_data + w_ic * loss_ic + w_ode * loss_ode
```

5. PIDL With A Missing Mechanism

PIDL should not learn the entire dynamics when a trusted mechanism is already known. In this example the known SIR right-hand side remains explicit, and a correction network $g_\omega(t, x)$ learns the missing nonlinear transfer.

$$\dot{x} = f_{\text{known}}(x; \beta, \gamma) + Bg_\omega(t, x). \tag{1}$$

What to inspect. The correction term should reduce residual error, but it should not become an unconstrained replacement for the known model. Inspect residual loss, correction regularization, and mean correction together.

6. Direct Control PINN

The direct control PINN trains a state network $\hat{x}_\theta(t)$ and a bounded control network $\hat{u}_\phi(t)$. It minimizes a malware-control objective while enforcing the controlled SIR dynamics.

$$\mathcal{L} = J(\hat{x}, \hat{u}) + w_{\text{res}}\|\dot{\hat{x}} - f(\hat{x}, \hat{u})\|^2 + w_{\text{ic}}\|\hat{x}(0) - x_0\|^2. \tag{2}$$

This method is simple and useful, but it is a direct optimization method. If a project claims PMP consistency, use the PMP-informed residuals below.

7. PMP-Informed PINN

The PMP-informed PINN adds a costate network $\hat{\lambda}_\psi(t)$ and enforces the optimality system rather than only the state equation.

$$\mathcal{L} = w_x\|\dot{x} - f(x, u)\|^2 + w_\lambda\|\dot{\lambda} + H_x(x, u, \lambda)\|^2 + w_u\|H_u(x, u, \lambda)\|^2 + w_b\mathcal{L}_{\text{boundary}}. \tag{3}$$

Why the stationarity residual matters. A small state residual says the trajectory satisfies the dynamics. A small stationarity residual gives stronger evidence that the learned control is compatible with PMP necessary conditions.

8. Training Diagnostics

The diagnostic figure generated by `scripts/run_training_iterations.py` has four panels:

- sparse-data inverse PINN: total loss and ODE residual;
- PIDL missing mechanism: residual loss and learned correction size;
- direct control PINN: objective, state residual, and mean control;
- PMP-informed PINN: state, costate, stationarity, and total residuals.

Start with `artifacts/experiments/OUTPUT_PREVIEW.md`, then open the CSV file behind the panel that matters for the method claim.

9. Baseline Comparisons

Training curves answer whether a loss is decreasing. Baseline comparisons answer whether the trained method is better than a simple alternative on a meaningful metric. The script therefore also writes `artifacts/experiments/baseline_comparison_metrics.csv` and `artifacts/figures/baseline_comparison.png`.

Topic	Baseline	Metric
Sparse-data inverse PINN	sparse interpolation and wrong-parameter SIR	full S/I/R mean squared error
PIDL missing mechanism	known SIR with the missing term removed	full S/I/R mean squared error
Controlled malware mitigation	no control, fixed controls, direct PINN, PMP-informed PINN, and rollout-optimized neural control	rollout objective and cumulative compromised devices

For control experiments, inspect both the training diagnostic and the rollout comparison. A control network can reduce its training loss while still needing independent ODE-rollout validation.

10. Extension Rules

1. If observations are noisy, add a validation split and report parameter uncertainty.
2. If the missing mechanism may depend on interventions, give the correction network access to (t, x, u), not only (t, x).
3. If a control should be deployable as feedback, use a control network $\pi(t, x)$ and train over multiple initial conditions.
4. If the model is high-dimensional, start with degree-level compartments before moving to full node-level or graph-neural PINNs.
5. If dynamics have impulses or jumps, train separate intervals or include jump residuals explicitly.

11. Minimal Reporting Checklist

A clear Note 2 experiment should report:

- model equations and true or assumed parameters;
- observed variables, observation times, and noise level;
- collocation sampling strategy;
- neural objects and architecture sizes;
- every loss term and its weight;
- learned parameters or learned controls;
- independent checks such as held-out data, ODE-solver validation, or comparison with FBSM/control baselines.